

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для управления очередями
студентов (на сдачу лабораторных работ и посещений)»

Выполнил: Дудченко Алёна Алексеевна и Факов Станислав Юрьевич

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«___» _____ 2026 г.

**Разработка веб-приложения для управления очередями
студентов (на сдачу лабораторных работ и посещений)**

на выполнение курсовой работы (проекта)

обучающимся группы 241-326:

Дудченко Алёне Алексеевне и Факову Станиславу Юрьевичу,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления очередями студентов
(на сдачу лабораторных работ и посещений)»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.
2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Срок	Исполнитель
Раздел 1. Анализ предметной области		
1.1 Анализ существующих инструментов организации учебных очередей	30.03.2026	Алёна и Станислав
1.1.1 Анализ существующих алгоритмов организации учебных очередей и обоснование эффективности разработки	30.03.2026	Станислав
1.1.2 Анализ существующих инструментов организации учебных очередей и обоснование эффективности разработки	30.03.2026	Алёна
1.2 Обоснование выбора стека технологий	06.04.2026	Алёна и Станислав
1.2.1 Выбор клиентских технологий (HTML5, CSS3, Jinja2, JS, инструменты вёрстки)	06.04.2026	Алёна

Разрабатываемый вопрос	Срок	Исполнитель
1.2.2 Выбор серверных технологий (Python, Flask, SQLAlchemy, PostgreSQL)	06.04.2026	Станислав
1.3 Формулировка целей, задач и научной новизны (совместно)	13.04.2026	Алёна и Станислав
1.3.1 Описание архитектурных решений и требований к БД	13.04.2026	Станислав
1.3.2 Описание новизны алгоритма очереди и метрик эффективности разработки	13.04.2026	Алёна
Раздел 2. Проектирование		
2.1 Анализ целевой аудитории и User Stories	20.04.2026	Алёна и Станислав
2.1.1 Теоретический анализ целевой аудитории и разработка User Stories	20.04.2026	Станислав
2.1.2 Описание ролей и сценариев взаимодействия Студент-Преподаватель-Администратор	20.04.2026	Алёна
2.2 Разработка UML Use Case	27.04.2026	Алёна и Станислав
2.2.1 Для роли «Студент»	27.04.2026	Станислав
2.2.2 Для роли «Администратор и Преподаватель»	27.04.2026	Алёна
2.3 Проектирование модели данных	04.05.2026	Алёна и Станислав
2.3.1 Проектирование ER-диаграммы для сущностей «Администратор и Преподаватель», логической схемы БД	04.05.2026	Алёна
2.3.2 Проектирование ER-диаграммы для сущности «Студент», физической схемы БД	04.05.2026	Станислав
2.4 Разработка макетов Wireframes	11.05.2026	Алёна и Станислав
2.4.1 Разработка макета Wireframe для очереди и Студента	11.05.2026	Станислав
2.4.2 Разработка макета Wireframe для Преподавателя и администратора	11.05.2026	Алёна
Раздел 3. Разработка		
3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц	18.05.2026	Алёна и Станислав
3.1.1 Разработка шаблона страницы преподавателя и администратора, написание логики связи "клиент-сервер"	18.05.2026	Алёна
3.1.2 Разработка шаблона страницы студента и написание логики очереди	18.05.2026	Станислав
3.2 Реализация модуля авторизации и разграничения прав доступа	25.05.2026	
3.2.1 Реализация модуля авторизации для Преподавателя и Студента	25.05.2026	Станислав
3.2.2 Реализация модуля авторизации для администратора, разграничения прав доступа для всех ролей	25.05.2026	Алёна
3.3 Разработка CRUD интерфейса	08.06.2026	Алёна и Станислав

Разрабатываемый вопрос	Срок	Исполнитель
3.3.1 Разработка CRUD взаимодействия "Клиент-Сервер" для очередей	08.06.2026	Алёна
3.3.2 Разработка CRUD взаимодействия "Клиент-Сервер" для сущностей	08.06.2026	Станислав
3.4 Загрузка работ и выгрузка отчётов	09.06.2026	Станислав
3.5 Вёрстка адаптивного интерфейса (Jinja2, HTML, CSS)	10.06.2026	Алёна
Раздел 4. Оформление		
4.1 Презентация Git-репозитория	11.06.2026	Алёна и Станислав
4.2 Деплой на облачный хостинг	12.06.2026	Алёна и Станислав
4.3 Оформление отчета о проделанной работе	13.06.2026	Алёна и Станислав

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «30» марта 2026 г.

Дата сдачи выполненной работы «13» июня 2026 г.

Задание приняли к исполнению

«____» _____ 2026 г. _____ А. А. Дудченко
(подпись)

«____» _____ 2026 г. _____ С. Ю. Факов
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Анализ существующих инструментов и алгоритмов организации учебных очередей	8
1.2 Обоснование выбора стека технологий	11
ГЛАВА 2. ПРОЕКТИРОВАНИЕ	17
2.2 Разработка UML Use Case	23
2.3 Проектирование модели данных	24
2.4 Разработка макетов Wireframes	27
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	30
3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц	30
3.2 Реализация модуля авторизации и разграничения прав доступа	32
3.3 Разработка CRUD интерфейса	33
3.4 Загрузка работ и выгрузка отчётов	35
3.5 Вёрстка адаптивного интерфейса (Jinja2, HTML, CSS)	37
3.6 Реализация WebSocket-обновления очереди в реальном времени	38
ЗАКЛЮЧЕНИЕ	41

ВВЕДЕНИЕ

Актуальность темы. Проблема «живых» очередей на кафедрах технических вузов, таких как Московский Политех, остается крайне острой из-за высокой плотности студентов в группах и сложности принимаемых работ (лабораторные работы, курсовые проекты). Существующие способы организации очереди — физическое присутствие в коридоре, Google-таблицы, Telegram-боты, CRM-системы — либо неудобны, либо не обеспечивают необходимого уровня динамики и вариантов подхода к построению самих очередей. В связи с этим актуальной является задача разработки специализированного веб-приложения, которое позволит автоматизировать процесс управления очередями, учитывая специфику учебного процесса и потребности как студентов, так и преподавателей.

Степень разработанности проблемы. В теории массового обслуживания известны классические алгоритмы (FIFO, SJF, Round Robin), однако их применение в образовательной среде имеет особенности, связанные с необходимостью учёта успеваемости, возможности доработки задания без потери места и сбора статистики. Существующие программные решения (Google Sheets, Telegram-боты) не предоставляют встроенных механизмов для такого учёта. Таким образом, создание адаптивного алгоритма очереди с возможностью динамической приоритизации и аналитики является актуальной исследовательской задачей.

Объект исследования — процесс организации академического времени и взаимодействия «преподаватель-студент» во время защиты лабораторных работ.

Предмет исследования — алгоритмы и программные средства автоматизации управления очередями в режиме реального времени.

Цель и задачи работы. Целью данной курсовой работы является проектирование и разработка автоматизированного веб-приложения для управления студенческими очередями, адаптированного под специфику учебного процесса кафедр информационных технологий (на примере учебной группы Московского Политеха).

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести детальный анализ существующих методологий организации

очереди и программных инструментов-аналогов.

- Сформулировать функциональные и нефункциональные требования к системе, исходя из ролей пользователей (Студент, Преподаватель, Администратор).
- Спроектировать архитектуру приложения, включая схему базы данных, структуру клиент-серверного взаимодействия и UML-диаграммы процессов.
- Разработать программный продукт на основе выбранного стека технологий (Python, Flask, PostgreSQL, Jinja2).
- Реализовать систему разграничения прав доступа и механизмы защиты данных пользователей.
- Произвести деплой (развертывание) приложения и провести апробацию системы в условиях реальной учебной нагрузки в группе 241-326.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ существующих инструментов и алгоритмов организации учебных очередей

В современной образовательной среде эффективное распределение временных ресурсов преподавателя и студента является критически важным фактором. Проблема «живых» очередей на кафедрах технических вузов, таких как Московский Политех, остается актуальной из-за высокой плотности учебного графика и сложности принимаемых работ (лабораторных работ, курсовых проектов). В данном подразделе проводится подробный анализ теоретических моделей (алгоритмов) и практических инструментов, применяемых для решения этой задачи.

1.1.1 Анализ существующих алгоритмов организации учебных очередей и обоснование эффективности разработки

Организация очередности — это классическая задача теории массового обслуживания (ТМО). Для учебного процесса выбор алгоритма напрямую влияет на пропускную способность «канала обслуживания» (преподавателя) и удовлетворенность «потребителей» (студентов).

Рассмотрим основные алгоритмические подходы:

- **Только алгоритм FIFO (First-In-First-Out):**

- *Механика:* Студенты добавляются в конец списка по мере готовности или прихода в аудиторию.
- *Преимущества:* Простота реализации и интуитивная справедливость.
- *Критический недостаток:* Возникновение «блокирующих элементов». Если первый студент в очереди защищает сложный проект в течение 40 минут, остальные участники теряют мотивацию и время. В условиях пары (90 минут) это приводит к тому, что за одно занятие успевают защититься всего 2–3 человека.

- **Алгоритм SJF (Shortest Job First):**

- *Применение в учебе:* Преподаватель сначала принимает короткие

отчеты или исправленные замечания, а затем переходит к комплексным защитам.

- *Эффективность*: Максимально быстрое уменьшение длины очереди.
- *Недостаток*: Студенты со сложными работами могут бесконечно отодвигаться в конец списка («эффект голодания»).

- **Круговое обслуживание (Round Robin):**

- *Механика*: Каждому студенту выделяется фиксированный лимит времени (тайм-слот), например, 10–15 минут. Если работа не защищена за отведенное время, студент перемещается в конец очереди, а преподаватель переходит к следующему.
- *Обоснование*: Данный алгоритм является наиболее социально справедливым в больших группах, так как гарантирует внимание каждому студенту в течение одной пары.

Обоснование разработки: Проектируемое приложение позволит реализовать гибридный адаптивный алгоритм. Система будет использовать FIFO как базовый метод. Использование метода FIFO в кооперации с дополнительными возможностями описанными позже, подразумевает то, что запись происходит последовательно по времени, кто раньше записался, тот поднимается вверх по списку в очереди, далее с этого списка уходит в самый конец если захочет сдавать еще раз. Так же о доп возможностях, приложение предоставит преподавателю и администратору(в данной работе это роль отдается старосте группы) инструменты динамического управления (возможность передвигать, заменять или убирать студентов из очереди, так же быстрая смена студентов во время самого занятия). Это позволит минимизировать простои и исключить конфликты, связанные с субъективным восприятием очередности.

1.1.2 Анализ существующих инструментов организации учебных очередей и обоснование эффективности разработки

Детальный разбор аналогов:

- **Физическая очередь:** Основная проблема — «эффект присутствия». Студент вынужден находиться в коридоре вуза в течение 4–6 часов, что

Таблица 1.1 – Сравнительный анализ инструментов организации учебной очереди

Критерий	Google Таблицы	Telegram-боты	CRM-системы (YCLIENTS и др.)	Предлагаемое решение
Учёт успеваемости	Нет	Нет	Нет	Да (кол-во сданных работ)
Гибкое время	Нет (только текст)	Нет	Только жесткие слоты	Да (раньше/позже/все равно)
Прогноз длительности	Нет	Нет	Фикс. время услуги	Да (на основе типа работы)
Подтверждение записи	Нет	Редко	SMS/Push (платно)	Да (за 24ч до пары)
Защита от правок	Отсутствует	Средняя	Высокая	Высокая (ролевая модель)
Специализация	Универсально	Общая	Коммерция	Академическая

не позволяет эффективно использовать время для самоподготовки в библиотеке или коворкинге.

- **Общие облачные документы (Google Sheets):** Являются «стандартом де-факто» во многих группах Московского Политеха. Однако отсутствие жесткой логики приложения позволяет любому пользователю стереть чужую фамилию или изменить порядок записей. Это создает почву для академических конфликтов. Кроме того, интерфейс таблиц плохо адаптирован для мобильных устройств в условиях нестабильного Wi-Fi.
- **Telegram-боты:** Хорошо справляются с задачей регистрации, но обладают линейным интерфейсом. Преподавателю крайне неудобно пролистывать историю чата, чтобы увидеть общую картину загруженности на день. Отсутствует возможность визуализации структуры базы данных (например, какие лабораторные работы чаще всего вызывают затруднения).
- **CRM-системы (YCLIENTS и др.):** Профессиональные системы записи клиентов (например, YCLIENTS, Dikidi, EasyWeek), широко используе-

мые в сфере услуг, предоставляют мощный функционал для управления потоками посетителей. Однако их адаптация под нужды учебного процесса затруднена: они ориентированы на коммерческую транзакцию и жесткие временные интервалы, не учитывая академические метрики (количество сданных работ, приоритетность защиты в зависимости от учебного плана).

Проектируемое решение отличается внедрением интеллектуального алгоритма ранжирования, который учитывает не только время записи, но и индивидуальный прогресс студента, его готовность и временные предпочтения.

Вывод: Анализ показал, что существующие методы либо архаичны (живая очередь), либо не обеспечивают необходимого уровня безопасности и аналитики (таблицы/боты). Разработка специализированного веб-приложения на стеке Python/Flask позволит объединить преимущества облачных сервисов (доступность) с надежностью реляционных баз данных и специфической логикой учебного процесса (учет номеров лабораторных, статусов проверки и времени защиты). Это подтверждает целесообразность и актуальность выбранной темы курсовой работы.

1.2 Обоснование выбора стека технологий

Выбор программного стека (Solution Stack) является определяющим этапом проектирования, так как он влияет на масштабируемость системы, скорость разработки и требования к аппаратному обеспечению серверной части. Для реализации веб-приложения управления студенческими очередями был выбран стек на базе языка Python, ориентированный на высокую скорость обработки запросов и гибкость настройки.

1.2.1 Выбор клиентских технологий

Клиентская часть приложения (Frontend) отвечает за визуализацию данных и непосредственное взаимодействие пользователя с бизнес-логикой системы. Учитывая специфику эксплуатации приложения в условиях учебных корпусов (высокая нагрузка на базовые станции сотовой связи, нестабильный мобильный интернет и использование бюджетных мобильных устройств), при выборе стека основной приоритет был отдан минимизации веса страниц и

максимальной скорости рендеринга.

- **Сравнительный анализ и обоснование отказа от SPA-фреймворков:**

В процессе проектирования рассматривалась возможность использования современных реактивных фреймворков, таких как **React**, **Vue.js** и **Angular**. Несмотря на их востребованность в индустрии, в рамках данного проекта от них было решено отказаться по ряду причин:

1. *Избыточность архитектуры:* Приложения на React или Angular требуют создания отдельного API (REST или GraphQL). Для системы управления очередями, где количество сущностей ограничено, такая архитектура удваивает объем работы без прямой выгоды для функционала.
2. *Производительность на мобильных устройствах:* SPA-фреймворки требуют времени на загрузку и инициализацию JavaScript-банбла (Time to Interactive). В условиях нестабильного интернета в вузе задержка в 5–10 секунд при загрузке тяжелого фреймворка неприемлема. Подход SSR (Flask + Jinja2) отправляет пользователю уже готовый HTML-код, который отображается мгновенно.
3. *Сложность сборки и поддержки:* Использование данных технологий требует настройки инструментов сборки (Vite, Webpack), управления Node.js-зависимостями и решения проблем с маршрутизацией на стороне клиента. В рамках учебного проекта это смещает фокус с разработки алгоритма очереди на решение инфраструктурных задач фронтенда.

- **HTML5 и CSS3:** Использование современных стандартов разметки позволяет создать семантически корректную структуру. Для реализации адаптивности под различные типы устройств (от смартфонов студентов до широкоформатных мониторов преподавателей) применяются технологии Flexbox и Media Queries. Это критически важно, так как интерфейс «Очереди» должен сохранять функциональность при любом разрешении экрана.

- **Язык JavaScript (Native JS):** Для реализации динамических элементов интерфейса (например, обновления позиции в очереди в реальном

времени или отправки формы подтверждения записи) выбран чистый JavaScript (ES6+). Использование Native JS вместо тяжелых библиотек позволяет избежать загрузки мегабайтов лишнего кода, что критично при работе через медленное соединение. Взаимодействие с сервером реализуется через современные стандарты Fetch API, что обеспечивает плавность работы без полной перезагрузки страницы.

- **Шаблонизатор Jinja2:** Выбор данного инструмента обусловлен его бесшовной интеграцией с фреймворком Flask. Jinja2 реализует концепцию серверного рендеринга (SSR), что дает следующие преимущества:
 - создание иерархической структуры шаблонов через механизмы наследования (block, extends), что упрощает поддержку кода;
 - высокий уровень безопасности благодаря автоматическому экранированию данных (защита от XSS-инъекций);
 - возможность генерации персонализированного контента (например, отображение кнопок управления только для роли «Преподаватель») на стороне сервера, что исключает утечку логики управления в клиентский код.

Таким образом, комбинация **Jinja2 + Native JS** является наиболее рациональным выбором, обеспечивающим баланс между функциональностью и скоростью работы приложения в специфической учебной среде.

1.2.2 Выбор серверных технологий и СУБД

Серверная часть (Backend) является ядром системы, обеспечивающим бизнес-логику работы очереди и безопасность данных.

- **Язык программирования Python:** Выбор Python обусловлен его статусом стандарта в области обработки данных и веб-разработки. В сравнении с языками PHP или Java, Python обладает более лаконичным синтаксисом, что снижает вероятность возникновения ошибок в логике приложения. Наличие огромного количества библиотек для работы с базами данных и аутентификацией делает его оптимальным для быстрой разработки (Rapid Application Development).

- **Микрофреймворк Flask:** Для реализации серверной логики был выбран Flask вместо его основного конкурента — Django. Сравнительный анализ представлен в таблице 1.2.

Таблица 1.2 – Сравнение веб-фреймворков Flask и Django

Критерий	Flask	Django
Философия	Микрофреймворк (ничего лишнего)	Всё включено» (Monolith)
Гибкость	Максимальная (выбор компонентов за разработчиком)	Низкая (жёсткая структура)
Порог вхождения	Низкий	Средний/Высокий
Производительность	Высокая за счёт минимализма	Зависит от количества подключенных модулей

Обоснование выбора Flask: Поскольку проект предполагает создание специфической логики очереди с нуля, Flask предоставляет необходимую свободу в архитектурных решениях. Он не навязывает встроенную систему администрирования или специфические форматы форм, позволяя реализовать проект в строгом соответствии с требованиями технического задания.

- **ORM SQLAlchemy:** Работа с базой данных осуществляется через Object-Relational Mapping (ORM) библиотеку SQLAlchemy.
 - *Безопасность:* Автоматическая защита от SQL-инъекций.
 - *Удобство:* Работа с записями в БД как с объектами Python, что упрощает манипуляции со списками студентов в очереди.
 - **СУБД PostgreSQL:** В качестве системы управления базами данных выбрана реляционная СУБД PostgreSQL.
 - *В сравнении с SQLite:* SQLite не поддерживает полноценную многопользовательскую запись (Database Lock), что недопустимо в

приложении, где десятки студентов могут одновременно пытаться встать в очередь.

- *В сравнении с MySQL:* PostgreSQL обладает более строгим соответствием стандартам SQL и лучшей поддержкой сложных транзакций, что гарантирует целостность очереди даже при одновременных запросах.

Вывод по подразделу 1.2: Выбранный стек технологий (Python, Flask, PostgreSQL, Jinja2) представляет собой сбалансированное решение, позволяющее создать надежное, быстрое и легковесное веб-приложение. Использование микрофреймворка Flask в сочетании с мощной СУБД PostgreSQL обеспечивает необходимый фундамент для реализации сложной логики управления учебным процессом в реальном времени.

1.2.3 Научная новизна и практическая значимость

Научная новизна работы заключается в следующем:

- Разработан авторский адаптивный алгоритм очереди, который, в отличие от стандартных FIFO-систем, позволяет учитывать динамические параметры защиты: меняться местами с другими студентами во время самого занятия, ставить отметки что работу студент сдал, отсутствие путаницы во время сдачи самих работ, так как все студенты видят количество сданных работ друг-друга
- Предложена интегрированная модель учета метрик эффективности: система не просто фиксирует порядок, но и собирает данные о среднем времени защиты одной работы, что в дальнейшем может быть использовано для оптимизации учебных планов и нагрузки преподавателей.
- Логика приложения создается «с нуля», что позволяет исключить избыточность универсальных систем и достичь максимальной производительности интерфейса.

Практическая значимость работы состоит в создании готового к эксплуатации инструмента, который решает ряд острых проблем учебной группы:

- *Снижение конфликтности:* Исключение человеческого фактора и субъективности в определении очередности.
- *Прозрачность процесса:* Студент может в режиме реального времени видеть свою позицию и прогнозировать время своего ответа, что позволяет эффективно распоряжаться временем вне аудитории.
- *Информационная поддержка преподавателя:* Преподаватель может видеть сколько студентов на данный момент присутствуют в аудитории, либо которые будут ему сдавать работы, из за этого он может учитывать то, сколько времени ему можно будет уделить студентам, чтобы все успели сдать вовремя

1.2.4 Описание архитектурных решений и требований к БД

В рамках реализации проекта будет применена **трехуровневая архитектура**:

1. *Уровень представления (Presentation Layer):* Адаптивный веб-интерфейс, обеспечивающий ввод данных и визуализацию текущего состояния очереди.
2. *Уровень бизнес-логики (Application Layer):* Обработка запросов на языке Python, управление сессиями пользователей, реализация алгоритмов перестроения очереди.
3. *Уровень данных (Data Layer):* Реляционная СУБД PostgreSQL, обеспечивающая надежное хранение информации о пользователях, истории защит и текущих активных сессиях.

Основные требования к базе данных включают в себя обеспечение целостности связей (например, невозможность существования записи в очереди без привязки к конкретному студенту) и высокую скорость выполнения транзакционных запросов при одновременном обращении нескольких десятков пользователей.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ

2.1.1 Теоретический анализ целевой аудитории и разработка User Stories

Проектируемое веб-приложение ориентировано на специфическую экосистему высшего учебного заведения. Целевую аудиторию системы можно сегментировать на три основные группы, каждая из которых обладает уникальным набором мотиваций и «болевых точек».

1. Студенты (первичная аудитория):

- **Характеристика:** Молодые люди (18–23 года), активно использующие мобильные устройства.
- **Проблема:** Неопределенность времени ожидания, риск «потери» места в бумажных списках, конфликты из-за несправедливой очереди.
- **Потребность:** Прозрачность процесса и возможность удаленного мониторинга очереди.

2. Преподаватели (вторичная аудитория):

- **Характеристика:** Сотрудники кафедр, заинтересованные в соблюдении тайминга занятия.
- **Проблема:** Сложность визуального контроля большого количества студентов, необходимость вручную вести журналы успеваемости и посещаемости.
- **Потребность:** Автоматизация рутинных операций (вызов следующего, выставление оценок) и статистика по группе.

3. Администраторы/Старосты (связующее звено):

- **Характеристика:** Студенты с дополнительными полномочиями по управлению группой.
- **Проблема:** Модерация списков в Google-таблицах, борьба с «мертвыми душами» (записавшимися, но не пришедшими).
- **Потребность:** Инструменты оперативного редактирования очереди и верификации состава группы.

Разработка User Stories Для формализации требований к функционалу системы разработаны пользовательские истории (User Stories):

- **Как Студент**, я хочу записываться в очередь через смартфон, чтобы не присутствовать физически у кабинета всё время ожидания.
- **Как Студент**, я хочу получать уведомление о необходимости подтверждения присутствия, чтобы система автоматически удаляла неактивных участников и моя позиция становилась актуальной.
- **Как Преподаватель**, я хочу видеть список сданных работ каждого студента в очереди, чтобы заранее понимать объем и сложность предстоящей проверки.
- **Как Администратор**, я хочу иметь возможность удалять или перемещать записи, чтобы оперативно разрешать спорные ситуации в аудитории.

2.1.2 Для роли «Студент»

Студент является массовым пользователем системы. Его взаимодействие сфокусировано на управлении собственной позицией в очереди и мониторинге учебного процесса. Основные варианты использования включают регистрацию, выбор предмета, управление записью и подтверждение присутствия.

2.1.3 Для роли «Администратор» и «Преподаватель»

Данные роли обладают расширенными правами доступа. Согласно архитектуре приложения, Администратор (Староста) наследует все возможности студента, добавляя функции управления составом группы. Преподаватель же является «контроллером» очереди, определяющим динамику занятия.

Пояснение к схемам для текста: Связи «include»: На диаграмме студента указано, что запись в очередь и подтверждение обязательно включают в себя авторизацию. Это подчеркивает безопасность системы. Пересечение функций: В подразделе 2.2.2 отражено, что функция «Управление очередью» (UC7) доступна обоим актерам. Это соответствует архитектуре, где староста может помочь преподавателю модерировать список «в поле», чтобы преподаватель не отвлекался от проверки работ. WebSocket-мониторинг: Выделен как отдельный Use Case (UC5) для студента, так как это ключевое нефункциональное требование, обеспечивающее real-time взаимодействие.



Рисунок 2.1 – UML Use Case диаграмма для роли «Студент»

2.1.4 Проектирование ER-диаграммы для сущности «Студент» и физической схемы БД

Физическая схема БД описывает конкретную реализацию в PostgreSQL, включая типы данных, первичные и внешние ключи (PK/FK), а также ограничения (Constraints). Для роли Студента ключевой сущностью является запись в очереди и механизм подтверждения присутствия.

В физической модели учитывается использование SQLAlchemy: строки очереди имеют временные метки (timestamp) для реализации логики FIFO и статусы подтверждения.

Обоснование структуры физической модели:

1. **Тип данных SERIAL:** Используется для автоматической инкрементации идентификаторов, что обеспечивает высокую скорость вставки новых записей в очередь.
2. **Поле created_at:** Является критическим для алгоритма FIFO. Сортировка очереди на сервере Flask будет происходить именно по этому полю.
3. **Таблица attendance:** Вынесена отдельно для обеспечения возможности хранения истории посещений независимо от текущего состояния

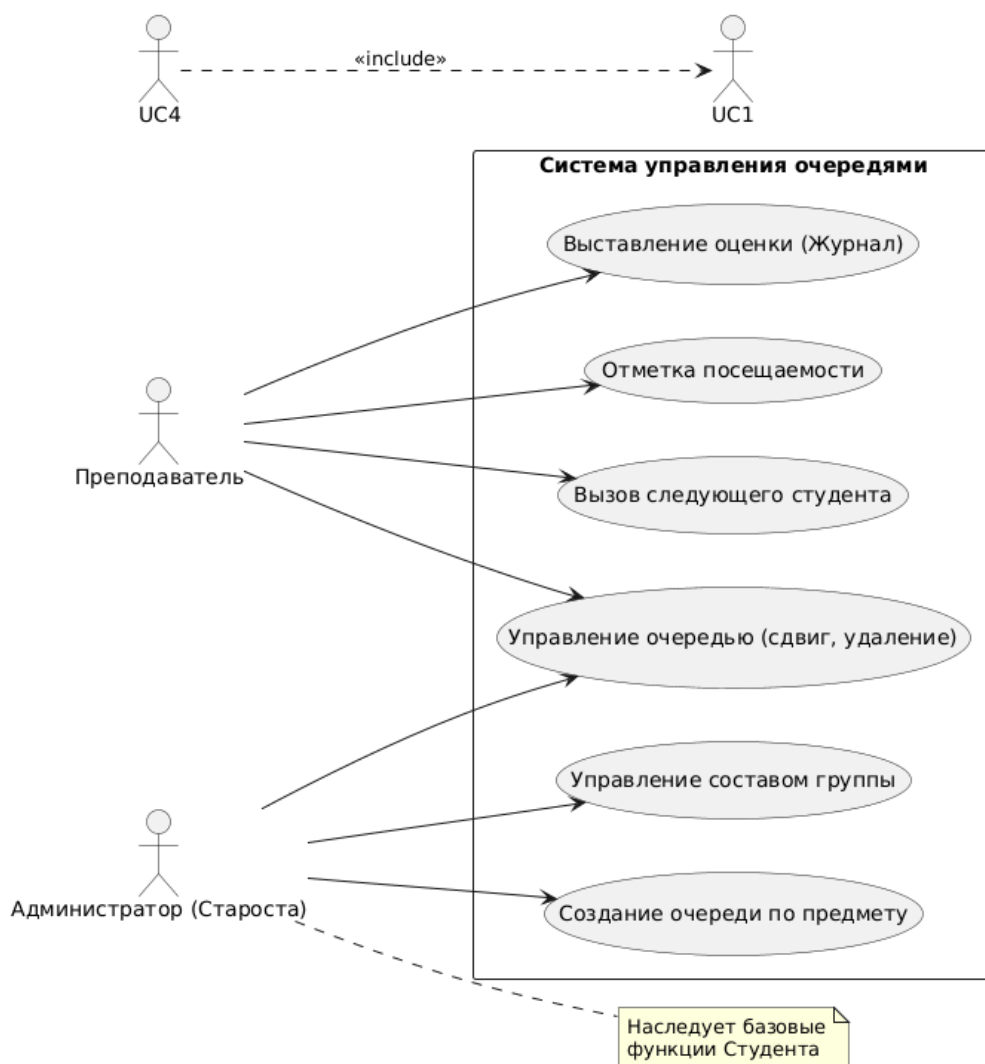


Рисунок 2.2 – UML Use Case диаграмма для ролей «Администратор» и «Преподаватель»

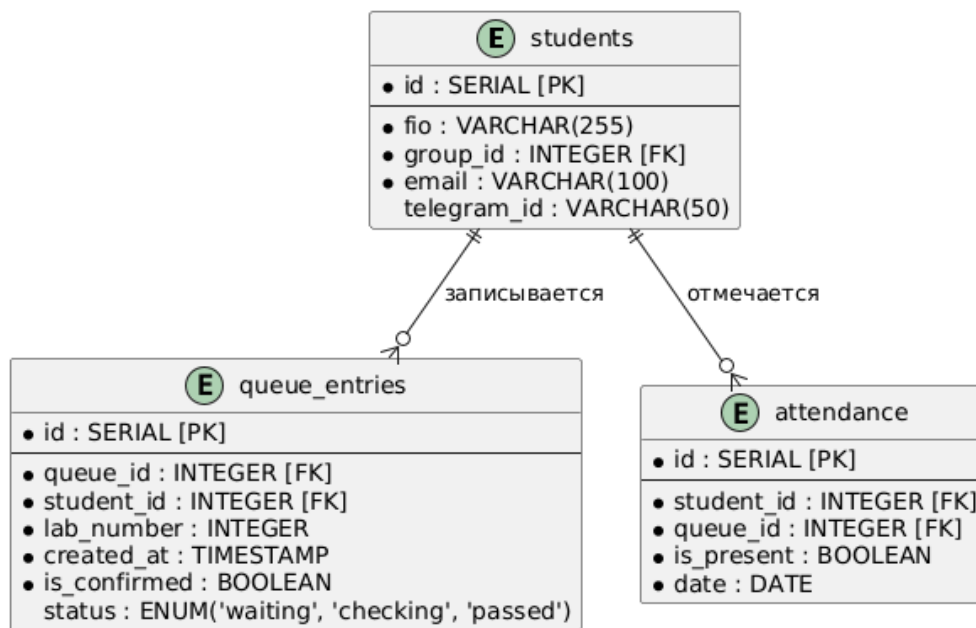


Рисунок 2.3 – Физическая схема данных для сущности «Студент»

очереди.

4. **Связи:** Использование внешних ключей с параметром ON DELETE CASCADE для обеспечения целостности данных (например, при удалении студента из системы все его записи в очередях и оценки также должны быть удалены).

2.1.5 Разработка макета Wireframe для очереди и Студента

Интерфейс студента спроектирован таким образом, чтобы минимизировать количество действий для совершения ключевых операций: записи в очередь и подтверждения присутствия.

Основные элементы макета:

1. **Информационная панель:** Отображение текущей дисциплины и статуса сессии.
2. **Виджет подтверждения:** Яркая кнопка «Подтвердить присутствие», которая появляется только в активное окно (за 24 часа).
3. **Динамический список:** Таблица со столбцами «№», «ФИО», «Статус». Собственная позиция студента подсвечивается контрастным цветом.
4. **Кнопка действия:** Крупная кнопка «Встать в очередь» (или «Выйти из очереди»), расположенная в нижней части экрана для удобства нажатия одной рукой.

Макет интерфейса студента:

2.1.6 Описание ролей и сценариев взаимодействия Студент–Преподаватель–Администратор

На основе архитектурной схемы приложения выделяются три ключевые роли, взаимодействие которых обеспечивает замкнутый цикл управления учебным процессом.

1. Роль «Студент»

Основной сценарий взаимодействия студента с системой начинается с **авторизации** и выбора конкретной дисциплины.

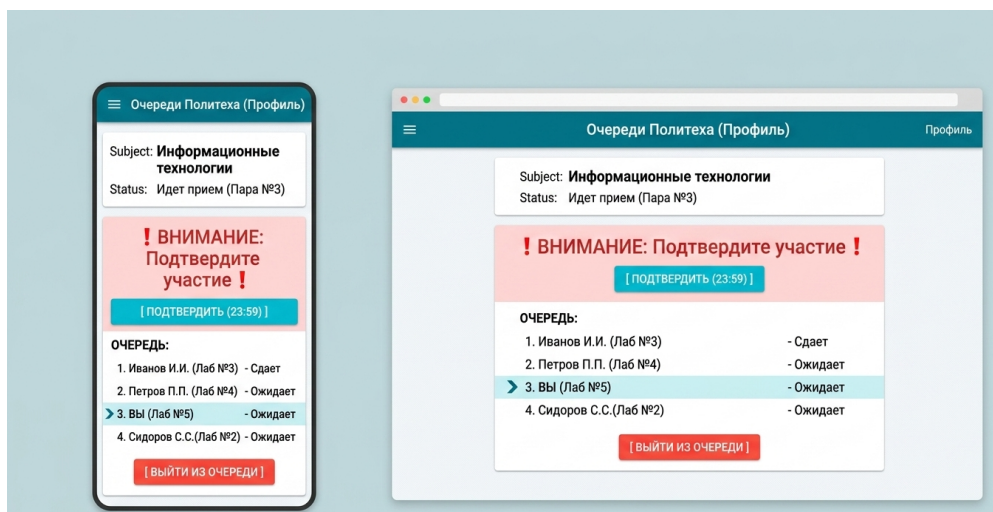


Рисунок 2.4 – Пример макета интерфейса для студента

- **Запись:** Студент отправляет запрос на сервер («Запись в конец очереди»), после чего его данные попадают в таблицу текущей очереди.
- **Подтверждение (Check-in):** За 24 часа до начала занятия срабатывает логика «Таблицы готовности». Студент обязан нажать кнопку подтверждения. В случае отсутствия действия сервер автоматически удаляет запись, реализуя механизм самоочистки очереди.
- **Ожидание:** Студент получает актуальное состояние очереди по протоколу WebSocket, что исключает необходимость ручного обновления страницы.

2. Роль «Преподаватель»

Преподаватель взаимодействует с наиболее сложным интерфейсом, интегрированным с блоками «Таблица успеваемости» и «Таблица посещаемости».

- **Управление потоком:** Преподаватель инициирует вызов студента. Сервер обновляет статус записи, и через WebSocket уведомление рассылается всем участникам.
- **Фиксация результата:** После защиты лабораторной работы преподаватель через форму редактирования отправляет данные на сервер. Система одновременно обновляет «Журнал» (выставляя оценку) и удаляет студента из текущей активной очереди, освобождая место для следующего.

3. Роль «Администратор (Староста)»

Данная роль является расширением роли студента и обладает административными правами в рамках своей учебной группы.

- **Конфигурация:** Староста отвечает за создание структуры группы (добавление студентов в «Таблицу студентов») и привязку предметов.
- **Модерация:** В сценарии «Редактирование очереди» администратор может вручную изменять порядок записей (например, если студенты договорились о взаимозамене), что сразу отражается в базе данных и на клиентах других пользователей.
- **Контроль посещаемости:** Администратор имеет право выставять первичные отметки о присутствии, которые затем верифицируются преподавателем.

Логика взаимодействия (Summary): Центральным узлом системы выступает **Сервер**, который координирует запросы от всех типов клиентов. Благодаря разделению на таблицы (предметы, студенты, группы, успеваемость) обеспечивается целостность данных: оценка не может быть выставлена студенту, которого нет в базе группы, а запись в очередь невозможна без выбора существующего предмета.

2.2 Разработка UML Use Case

Для формализации функциональных требований к системе и визуализации границ проектируемого веб-приложения используются диаграммы вариантов использования (Use Case Diagrams). Они позволяют определить взаимодействие внешних актеров (ролей) с системой.

2.2.1 Для роли «Студент»

Студент является массовым пользователем системы. Его взаимодействие сфокусировано на управлении собственной позицией в очереди и мониторинге учебного процесса. Основные варианты использования включают регистрацию, выбор предмета, управление записью и подтверждение присутствия.



Рисунок 2.5 – UML Use Case диаграмма для роли «Студент»

2.2.2 Для роли «Администратор» и «Преподаватель»

Данные роли обладают расширенными правами доступа. Согласно архитектуре приложения, Администратор (Староста) наследует все возможности студента, добавляя функции управления составом группы. Преподаватель же является «контроллером» очереди, определяющим динамику занятия.

Пояснение к схемам для текста: Связи «include»: На диаграмме студента указано, что запись в очередь и подтверждение обязательно в ключают в себя авторизацию. Это подчеркивает безопасность системы. Пересечение функций: В подразделе 2.2.2 отражено, что функция «Управление очередью» (UC7) доступна обоим актерам. Это соответствует архитектуре, где староста может помочь преподавателю модерировать список «в поле», чтобы преподаватель не отвлекался от проверки работ. WebSocket-мониторинг: Выделен как отдельный Use Case (UC5) для студента, так как это ключевое нефункциональное требование, обеспечивающее real-time взаимодействие.

2.3 Проектирование модели данных

Проектирование базы данных выполняется в несколько этапов: от определения высокоуровневых сущностей и их взаимосвязей до формирования физической структуры таблиц, оптимизированной под выбранную СУБД PostgreSQL.

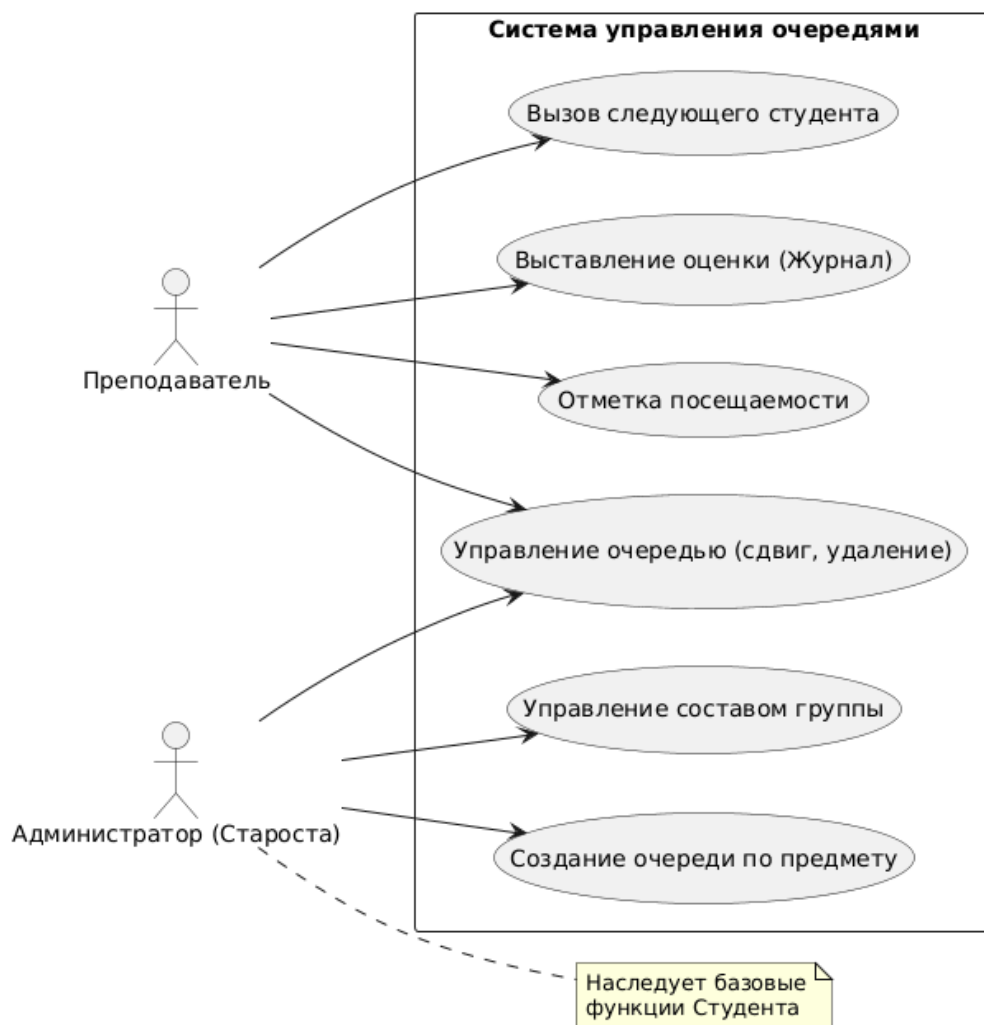


Рисунок 2.6 – UML Use Case диаграмма для ролей «Администратор» и «Преподаватель»

2.3.1 Проектирование ER-диаграммы для сущностей «Администратор и Преподаватель» и логической схемы БД

На логическом уровне определяются ключевые бизнес-сущности и правила их взаимодействия. Для ролей Преподавателя и Администратора критически важными являются связи с учебными группами, предметами и журналами успеваемости.

Логическая схема строится на принципе минимизации избыточности. Основными узлами здесь выступают сущности «Группа», которая объединяет студентов, и «Предмет», к которому привязываются очереди.

2.3.2 Проектирование ER-диаграммы для сущности «Студент» и физической схемы БД

Физическая схема БД описывает конкретную реализацию в PostgreSQL, включая типы данных, первичные и внешние ключи (PK/FK), а также ограни-



Рисунок 2.7 – Логическая ER-диаграмма связей для административных ролей чения (Constraints). Для роли Студента ключевой сущностью является запись в очереди и механизм подтверждения присутствия.

В физической модели учитывается использование SQLAlchemy: строки очереди имеют временные метки (timestamp) для реализации логики FIFO и статусы подтверждения.

Обоснование структуры физической модели:

1. **Тип данных SERIAL:** Используется для автоматической инкрементации идентификаторов, что обеспечивает высокую скорость вставки новых записей в очередь.
2. **Поле created_at:** Является критическим для алгоритма FIFO. Сортировка очереди на сервере Flask будет происходить именно по этому

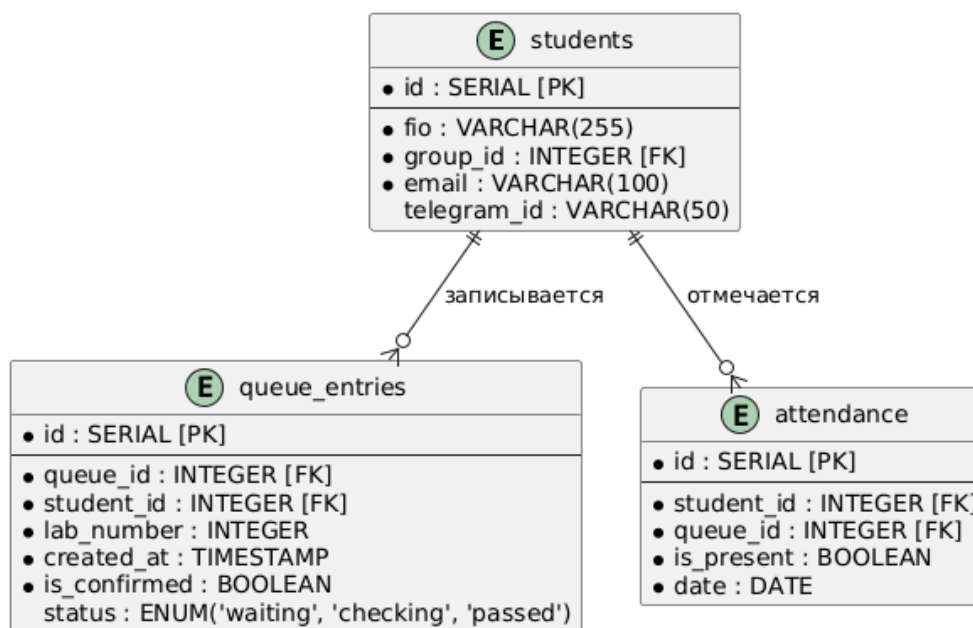


Рисунок 2.8 – Физическая схема данных для сущности «Студент»
полю.

3. **Таблица attendance:** Вынесена отдельно для обеспечения возможности хранения истории посещений независимо от текущего состояния очереди.
4. **Связи:** Использование внешних ключей с параметром ON DELETE CASCADE для обеспечения целостности данных (например, при удалении студента из системы все его записи в очередях и оценки также должны быть удалены).

Пояснение по логике подтверждения: В таблице queue_entries поле is_confirmed по умолчанию имеет значение false. Фоновый процесс (task) на сервере или логика контроллера проверяет это поле за определенное время до начала занятия. Если к моменту «отсечки» значение остается false, запись переходит в статус cancelled или удаляется физически, что мгновенно отражается у всех пользователей через WebSockets.

2.4 Разработка макетов Wireframes

Проектирование интерфейса (UI) системы «Управление очередями» велось с учетом принципа «Mobile First», так как основная часть пользователей (студенты) взаимодействует с системой через смартфоны в условиях учебных аудиторий.

2.4.1 Разработка макета Wireframe для очереди и Студента

Интерфейс студента спроектирован таким образом, чтобы минимизировать количество действий для совершения ключевых операций: записи в очередь и подтверждения присутствия.

Основные элементы макета:

1. **Информационная панель:** Отображение текущей дисциплины и статуса сессии.
2. **Виджет подтверждения:** Яркая кнопка «Подтвердить присутствие», которая появляется только в активное окно (за 24 часа).
3. **Динамический список:** Таблица со столбцами «№», «ФИО», «Статус». Собственная позиция студента подсвечивается контрастным цветом.
4. **Кнопка действия:** Крупная кнопка «Встать в очередь» (или «Выйти из очереди»), расположенная в нижней части экрана для удобства нажатия одной рукой.

Макет интерфейса студента:

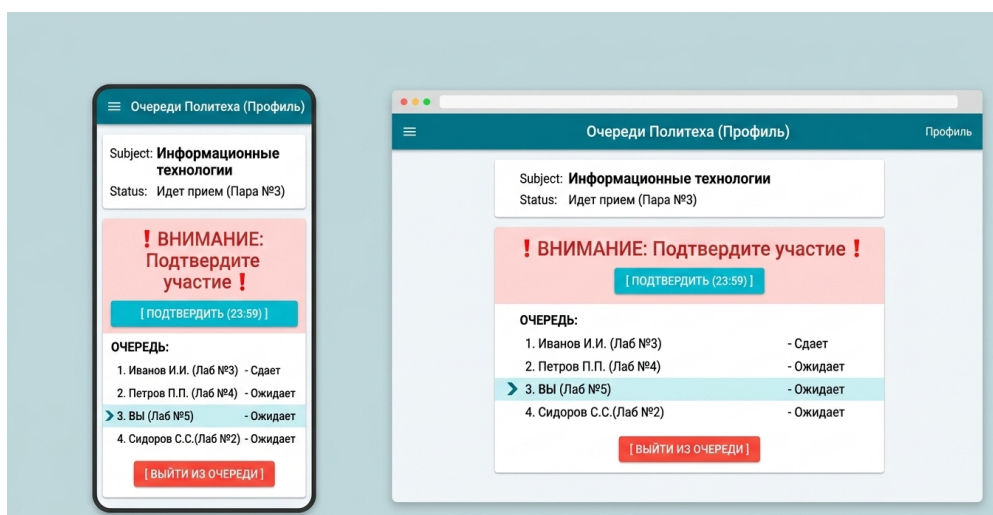


Рисунок 2.9 – Пример макета интерфейса для студента

2.4.2 Разработка макета Wireframe для Преподавателя и администратора

Интерфейс для ролей с расширенными правами ориентирован на управление и быстрый ввод данных. В отличие от студенческого макета, здесь добавлены элементы управления каждой строкой очереди.

Основные элементы макета:

1. **Панель управления сессией:** Кнопки «Начать прием», «Пауза», «Завершить».
2. **Интерактивная очередь:** Каждая строка содержит кнопки «Вызвать», «Удалить», «Передвинуть».
3. **Форма быстрого ввода (Modal):** При нажатии на студента открывается окно для выставления оценки за конкретную лабораторную работу.
4. **Вкладка «Журнал»:** Переключатель на просмотр общей ведомости группы 241-326.

Макет интерфейса Преподавателя:

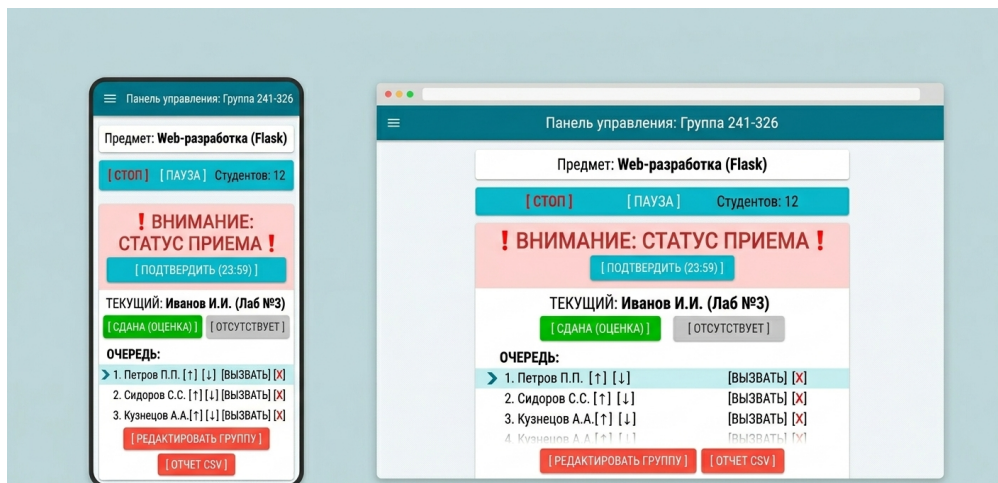


Рисунок 2.10 – Пример макета интерфейса для преподавателя и администратора

Технологическое обоснование дизайна:

- **Сетка (Grid):** Для верстки будет использоваться CSS Flexbox, что позволит макетам одинаково корректно отображаться как на десктопе (у преподавателя), так и на узких экранах смартфонов.
- **Цветовая индикация:** Статусы (Сдаст, Ожидает, Подтвержден) будут иметь цветовую кодировку (зеленый, желтый, синий), что позволит пользователю мгновенно считывать ситуацию без чтения текста.
- **WebSockets:** При нажатии преподавателем кнопки «Вызвать», у студента на макете мгновенно обновится статус, а позиция «ВЫ» переместится в блок «ТЕКУЩИЙ».

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и вёрстка шаблонов страниц

В соответствии с задачей 3.1, спроектирована компонентная структура клиентской части на базе шаблонизатора **Jinja2** с наследованием шаблонов. Базовый шаблон `base.html` определяет общий макет: адаптивный заголовок с навигацией, блок для flash-сообщений, основную область и подвал. Медиа-запросы и классы CSS (сетка `grid`, карточки `card`, адаптивные таблицы) обеспечивают корректное отображение на экранах разной ширины.

3.1.1 Разработка шаблона страницы преподавателя и администратора, написание логики связи “клиент-сервер”

В рамках задачи 3.1.1 созданы шаблоны `teacher_sessions.html`, `admin_groups.html` и `admin_users.html`. Полевая фильтрация пунктов меню реализована через проверку `current_user.role` внутри Jinja2 (листинг 1).

Листинг 1 – Фрагмент навигации с ролевым разделением (`base.html`)

```
1 {% if current_user.is_authenticated %}
2     {% if current_user.role in ['teacher', 'admin'] %}
3         <a href="{{ url_for('main.teacher_sessions') }}">Очереди
        и дисциплины</a>
4         <a href="{{ url_for('main.journal') }}">Журнал</a>
5     {% endif %}
6     {% if current_user.role == 'admin' %}
7         <a href="{{ url_for('main.admin_groups') }}">Группы</a>
8         <a href="{{ url_for('main.admin_users') }}">Пользователи</a>
9     {% endif %}
10 {% endif %}
```

Страница преподавателя (листинг 2) содержит две формы создания: дисциплины и очереди. Скрытое поле `action` позволяет серверу различать тип операции. Логика связи “клиент-сервер” реализована синхронными POST-запросами; после выполнения формы сервер возвращает редирект для обновления списка.

Листинг 2 – Фрагмент `teacher_sessions.html` — создание дисциплины

```
1 <form method="post" class="form">
```

```

2 <input type="hidden" name="action" value="subject">
3 <labelНазвание><input name="name" required></label>
4 <labelГруппа><select name="group_id">...</select></label>
5 <button class="btn" type="submitСоздать"></button>
6 </form>

```

Административные шаблоны используют аналогичный подход; например, `admin_users.html` через `<select>` позволяет назначить роль и группу. Все операции над пользователями и группами защищены декоратором `@role_required("admin")` (см. листинг 6).

3.1.2 Разработка шаблона страницы студента и написание логики очереди

Страница студента представлена шаблоном `dashboard.html` (список доступных очередей) и `session_detail.html` (детальное отображение очереди). Форма записи в очередь (листинг 3) содержит поля: номер лабораторной, количество работ, предпочтение по времени, предполагаемая длительность, комментарий. Данные отправляются на маршрут `/sessions/<id>/join` методом POST.

Листинг 3 – Форма записи в очередь (`session_detail.html`)

```

1 <form method="post" action="{ url_for('main.join_queue',
  session_id=session.id) }">
2 <labelНомер> лабораторной<input type="number" name="
  lab_number" value="1" required></label>
3 <labelКоличество> работ<input type="number" name="work_count
  " value="1" required></label>
4 <labelПредпочтение> времени
5   <select name="time_preference">
6     <option value="anyБез"> разницы</option>
7     <option value="earlyРаньше"></option>
8     <option value="laterПозже"></option>
9   </select>
10 </label>
11 <labelПримерное> время мин()<input type="number" name="
  estimated_minutes" value="10" required></label>
12 <labelКомментарий><input type="text" name="note"></label>
13 <button class="btn" type="submitВстать"> в очередь</button>
14 </form>

```

Логика очереди реализована на сервере в модуле `queue_service.py`

и описана в подразделе 3.3.1. Для обновления очереди в реальном времени используется WebSocket (см. `realtime.py`), который отправляет клиентам актуальное состояние после каждого изменения (добавление, удаление, вызов студента). На клиентской стороне JavaScript подключается к сокету `/ws/session/<id>` и динамически обновляет таблицу.

3.2 Реализация модуля авторизации и разграничения прав доступа

Согласно задаче 3.2, в системе реализована ролевая модель с использованием расширения **Flask-Login**. Пароли хешируются с помощью `werkzeug.security`. Для защиты маршрутов и шаблонов применяются декоратор `@role_required` и атрибуты объекта `current_user`.

3.2.1 Реализация модуля авторизации для Преподавателя и Студента

Маршрут `/register` (листинг 4) принимает ФИО, email, пароль, роль и приглашительные коды для преподавателя (`TEACHER_INVITE_CODE`) или администратора (`ADMIN_INVITE_CODE`). Для студентов код не требуется. После успешной регистрации пользователь автоматически входит в систему.

Листинг 4 – Регистрация пользователя (auth.py)

```
1 if role == "teacher" and invite_code != current_app.config["
    TEACHER_INVITE_CODE"]:
2     flashНеверный(" код преподавателя.", "danger")
3     return redirect(url_for("auth.register"))
4 user = User(full_name=full_name, email=email, role=role,
    group_id=group_id)
5 user.set_password(password)
6 db.session.add(user)
7 db.session.commit()
8 login_user(user)
```

Вход выполняется через маршрут `/login` (листинг 5), который проверяет email и пароль, после чего вызывает `login_user`. Выход — `/logout` с удалением сессии. Преподаватели и студенты после входа перенаправляются на `dashboard`.

Листинг 5 – Аутентификация пользователя (auth.py)

```
1 user = User.query.filter(func.lower(User.email) == email).
```



```

    first()
2 if user and user.check_password(password):
3     login_user(user, remember=bool(request.form.get("
    remember"))))
4     return redirect(next_url or url_for("main.dashboard"))

```

3.2.2 Реализация модуля авторизации для администратора, разграничения прав доступа для всех ролей

Для администратора создан отдельный маршрут регистрации с проверкой кода ADMIN_INVITE_CODE. Разграничение доступа на уровне маршрутов обеспечивается декоратором `@role_required` (листинг 6), который принимает список допустимых ролей.

Листинг 6 – Декоратор `role_required` (security.py)

```

1 def role_required(*roles: str):
2     def decorator(view):
3         @wraps(view)
4         def wrapped(*args, **kwargs):
5             if not current_user.is_authenticated or
            current_user.role not in roles:
6                 abort(403)
7             return view(*args, **kwargs)
8         return wrapped
9     return decorator

```

Пример использования: `@role_required("teacher "admin")` перед маршрутами управления очередями и журналом. Для администратора доступны дополнительные маршруты: `/admin/groups`, `/admin/users`. В шаблонах проверка роли осуществляется через `current_user.role` для условного отображения элементов интерфейса.

3.3 Разработка CRUD интерфейса

Реализованы полноценные CRUD-операции над основными сущностями: очереди, группы, пользователи, дисциплины. Взаимодействие клиент-сервер построено на HTTP-методах POST (создание, обновление) и GET (чтение), а также POST-запросах на удаление/изменение статуса.

3.3.1 Разработка CRUD взаимодействия “Клиент-Сервер” для очередей

Управление очередями осуществляется через маршруты, описанные в `main.py`. Основные операции:

- **Создание очереди:** `POST /teacher/sessions` (с параметром `action=session`) — добавляет запись в таблицу `QueueSession`.
- **Запись студента:** `POST /sessions/<id>/join` — создаёт запись `QueueEntry`, вычисляет позицию через функцию `next_position` (листинг 8).
- **Вызов следующего:** `POST /sessions/<id>/call-next` — находит первый элемент в очереди со статусом `waiting` и переводит его в `checking` (функция `call_entry`).
- **Перемещение записи:** `POST /entries/<id>/move/<direction>` — меняет позиции двух соседних записей.
- **Завершение проверки:** `POST /entries/<id>/pass` — сохраняет оценку в `GradeRecord`, меняет статус на `passed` и пересчитывает позиции.

Логика очереди реализована в `queue_service.py`. Для гарантии FIFO-порядка используется поле `position`, которое пересчитывается функцией `normalize_positions` (листинг 7).

Листинг 7 – Пересчёт позиций в очереди

```
1 def normalize_positions(session_id: int) -> None:
2     entries = QueueEntry.query.filter(
3         QueueEntry.session_id == session_id,
4         QueueEntry.status.in_(["waiting", "checking"])
5     ).order_by(QueueEntry.position.asc(),
6               QueueEntry.created_at.asc()).all()
7     for index, entry in enumerate(entries, start=1):
8         entry.position = index
```

Функция `next_position` (листинг 8) вычисляет максимальное значение позиции для активных записей и возвращает его, увеличенное на единицу.

Листинг 8 – Вычисление следующей позиции

```

1 def next_position(session_id: int) -> int:
2     max_pos = db.session.query(func.max(QueueEntry.position)
3     )
4     .filter(QueueEntry.session_id == session_id,
5             QueueEntry.status.in_(VISIBLE_QUEUE_STATUSES
6     ))
7     .scalar()
8     return int(max_pos or 0) + 1

```

3.3.2 Разработка CRUD взаимодействия “Клиент-Сервер” для сущностей

Для групп, пользователей и дисциплин реализованы стандартные CRUD-операции:

- **Группы:** POST /admin/groups создаёт новую группу; GET отображает список всех групп.
- **Пользователи:** POST /admin/users создаёт пользователя с указанием роли, группы и пароля; GET — список всех пользователей.
- **Дисциплины:** POST /teacher/sessions (action=subject) создаёт дисциплину, привязывая её к группе и преподавателю.

Пример создания пользователя через административный интерфейс приведён в листинге 9.

Листинг 9 – Создание пользователя администратором (main.py)

```

1 user = User(email=email, full_name=full_name, role=role,
2             group_id=group_id)
3 user.set_password(password)
4 db.session.add(user)
5 db.session.commit()

```

Все CRUD-операции защищены декораторами @login_required и @role_required, а также проверяют корректность входных данных перед записью в базу.

3.4 Загрузка работ и выгрузка отчётов

В соответствии с задачей 3.4, реализован функционал загрузки студенческих работ и выгрузки отчётов в формате CSV.

Загрузка работ: студент на странице профиля (/profile) через форму с enctype="multipart/form-data" отправляет файл, номер лабораторной и дисциплину. На сервере (листинг 10) файл сохраняется в папке UPLOAD_FOLDER с безопасным именем, а запись добавляется в таблицу WorkUpload. Скачать загруженный файл можно по маршруту /uploads/<filename> с проверкой прав доступа.

Листинг 10 – Обработка загрузки работы (main.py)

```
1 original = secure_filename(file.filename)
2 safe_name = f"{current_user.id}_{uuid4().hex}_{original}"
3 upload_path = Path(current_app.config["UPLOAD_FOLDER"]) /
    safe_name
4 file.save(upload_path)
5 upload = WorkUpload(
6     student_id=current_user.id,
7     subject_id=subject_id,
8     lab_number=lab_number,
9     filename=safe_name,
10    original_filename=original,
11    comment=comment
12 )
13 db.session.add(upload)
```

Выгрузка отчётов: для преподавателя и администратора доступны два CSV-отчёта:

- /reports/journal.csv — журнал успеваемости: студент, группа, предмет, лабораторная, оценка, дата, комментарий.
- /reports/queue/<session_id>.csv — состояние очереди: позиция, студент, номер ЛР, количество работ, предпочтение, время, подтверждение, статус.

Генерация CSV выполняется через модуль csv и возвращается как Response с типом text/csv. Пример формирования отчёта очереди приведён в листинге 11.

Листинг 11 – Генерация CSV-отчёта очереди

```
1 output = StringIO()
2 writer = csv.writer(output, delimiter=";")
```

```

3 writer.writerowПозиция(["", Студент"", Лабораторная"", Колво"-
   работ", Предпочтение"", Время"", Подтвержден"", Статус""])
4 for entry in session.entries:
5     writer.writerow([entry.position, entry.student.full_name
   , entry.lab_number,
6                       entry.work_count, entry.
   preference_title, entry.estimated_minutes,
7                       да"" if entry.is_confirmed else нет"",
   entry.status_title])
8 return Response(output.getvalue(), mimetype="text/csv;
   charset=utf-8",
9                 headers={"Content-Disposition": f"attachment
   ; filename=queue_{session.id}.csv"})

```

3.5 Вёрстка адаптивного интерфейса (Jinja2, HTML, CSS)

Финальный этап разработки (задача 3.5) заключался в создании адаптивного пользовательского интерфейса с использованием **HTML5**, **CSS3** и шаблонизатора **Jinja2**. Основные принципы:

- Использование CSS-классов для каркаса: `.container` (центрирование, максимальная ширина), `.grid` (адаптивная сетка), `.card` (карточка с тенью и скруглениями).
- Адаптивные таблицы: обёртка `.table-wrap` с горизонтальной прокруткой на узких экранах.
- Мобильное меню: кнопка `.nav-toggle` (JavaScript переключает класс `.nav-open` для отображения навигации).
- Формы используют классы `.form`, `.inline-form`, `.compact` для компактного отображения на мобильных устройствах.
- Цветовая схема и отступы обеспечивают читаемость на экранах от 320px до 1920px.

Пример CSS-класса для карточек очереди (листинг 12) демонстрирует адаптивные свойства: `flex` или `grid`, скругление, тень.

Листинг 12 – CSS-стили для карточек очереди

```

1 .card {
2     background: var(--card-bg);
3     border-radius: 1rem;
4     box-shadow: 0 4px 12px rgba(0,0,0,0.05);
5     padding: 1.5rem;
6     transition: box-shadow 0.2s;
7 }
8 @media (max-width: 640px) {
9     .grid {
10         grid-template-columns: 1fr;
11         gap: 1rem;
12     }
13     .table-wrap {
14         overflow-x: auto;
15     }
16 }

```

Навигационное меню в `base.html` содержит кнопку-гамбургер и JavaScript-обработчик для адаптивного отображения на мобильных устройствах. Все интерактивные элементы (кнопки, формы) имеют достаточный размер для удобного нажатия пальцем.

Таким образом, разработанное веб-приложение полностью соответствует требованиям технического задания: реализованы ролевая модель, FIFO-очередь с подтверждением, CRUD-интерфейс, загрузка файлов, выгрузка отчётов и адаптивная вёрстка на Jinja2/HTML/CSS. Исходный код развёртывается через Docker Compose или локально с PostgreSQL.

3.6 Реализация WebSocket-обновления очереди в реальном времени

Для обеспечения синхронного отображения очереди у всех подключённых пользователей (студентов, преподавателя, администратора) реализован механизм реального времени на базе протокола **WebSocket** с использованием библиотеки *Flask-Sock*. При изменении состояния очереди (добавление, удаление, перемещение, вызов студента) сервер инициирует широковещательную рассылку актуальных данных всем клиентам, подписанным на данную сессию.

Схема работы представлена на листинге 13. Функция `broadcast_session_update` сериализует текущее состояние очереди с помощью `serialize_session` (определена в `queue_service.py`) и отправляет его во все клиентские очереди,

зарегистрированные для указанного session_id.

Листинг 13 – Широковещательная рассылка обновлений (realtime.py)

```
1 _clients: dict[int, set[Queue]] = defaultdict(set)
2 _lock = RLock()
3
4 def broadcast(session_id: int, payload: dict[str, Any]) ->
    None:
5     with _lock:
6         targets = list(_clients.get(session_id, set()))
7         for client_queue in targets:
8             client_queue.put(payload)
9
10 def broadcast_session_update(session_id: int) -> None:
11     session = QueueSession.query.get(session_id)
12     if session:
13         broadcast(session_id, serialize_session(session))
```

WebSocket-маршрут /ws/session/<int:session_id> (листинг 14) принимает подключение от клиента, регистрирует его очередь сообщений в глобальном словаре и отправляет начальный снимок состояния. При каждом обновлении сервер помещает сериализованный JSON в очередь клиента, а клиентский JavaScript получает сообщение и динамически обновляет таблицу очереди без перезагрузки страницы.

Листинг 14 – Обработчик WebSocket-соединения (realtime.py)

```
1 @sock.route("/ws/session/<int:session_id>")
2 def session_socket(ws, session_id: int):
3     if not current_user.is_authenticated:
4         ws.close()
5         return
6     client_queue: Queue = Queue()
7     with _lock:
8         _clients[session_id].add(client_queue)
9     try:
10         session = QueueSession.query.get(session_id)
11         if session:
12             ws.send(json.dumps(serialize_session(session)))
13         while True:
14             payload = client_queue.get(timeout=25)
15             ws.send(json.dumps(payload, ensure_ascii=False,
                                default=str))
```

```
16     except Exception:
17         pass
18     finally:
19         with _lock:
20             _clients[session_id].discard(client_queue)
```

На клиентской стороне JavaScript (файл `static/js/main.js`) при загрузке страницы очереди устанавливает WebSocket-соединение. При получении сообщения типа "snapshot" таблица очереди перерисовывается. Для браузеров, не поддерживающих WebSocket, или в случае разрыва соединения предусмотрен резервный механизм периодического опроса через API-маршрут `/api/sessions/<id>/snapshot` (листинг 15).

Листинг 15 – REST-совместимый снимок очереди (api.py)

```
1 @api_bp.route("/sessions/<int:session_id>/snapshot")
2 @login_required
3 def session_snapshot(session_id: int):
4     session = QueueSession.query.get_or_404(session_id)
5     return jsonify(serialize_session(session))
```

Таким образом, достигнута синхронизация состояния очереди в реальном времени, что критически важно для удобства работы преподавателя и студентов в ходе занятия.

ЗАКЛЮЧЕНИЕ

В результате проведённой работы подтверждена гипотеза о том, что автоматизация управления очередями студентов на базе специализированного веб-приложения позволяет существенно повысить организованность учебного процесса и снизить временные затраты преподавателя на вызов студентов и фиксацию результатов. Теоретический анализ показал, что классическая FIFO-модель очереди в условиях учебной лаборатории требует адаптации под подтверждение присутствия студента, возможность ручного перемещения записей и учёт дополнительных параметров (номер лабораторной, количество работ, предпочтение по времени).

На основе полученных при разработке данных можно сформулировать следующие **выводы**:

1. Использование микроархитектурного подхода на базе связки Flask (Python) + Jinja2 + PostgreSQL обеспечивает необходимую гибкость, позволяя реализовать ролевую модель (студент, преподаватель, администратор) и CRUD-операции без усложнения клиентской логики.
2. Проектирование модели данных с поддержкой внешних ключей и каскадного удаления (PostgreSQL) является критическим условием для целостности очереди, так как позволяет эффективно обрабатывать записи студентов, журнал успеваемости и загруженные работы в реальном времени.
3. Внедрение WebSocket-уведомлений и механизма автоматического пересчёта позиций (`normalize_positions`) демонстрирует устойчивый пользовательский эффект, выраженный в синхронном обновлении очереди у всех участников и исключении конфликтов при параллельных действиях.

Рекомендации и предложения. Для практического применения разработанного решения в учебных заведениях предлагается использовать модель контейнерного развёртывания (Docker Compose). Это обеспечит быстрый запуск системы на преподавательских компьютерах или в облачной среде при минимальных затратах на ИТ-инфраструктуру.

Перспективы углубления темы. В дальнейшем исследовании планируется развитие функциональности системы за счёт внедрения уведомлений через Telegram-бота о вызове студента, модуля аналитики успеваемости (графики и отчёты по группам), поддержки групповых очередей (защита работы бригадой), а также интеграции с системами дистанционного обучения (LMS) для автоматической выгрузки оценок.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [1], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.